Speech and Audio Recognition

Midterm Review

소리의 표본화부터 전사 없는 표현 학습까지 연결한다.

Midterm Review · Lecture 2–5
Signal → Spectrum → Alignment → Representation

Exam Map

각 장은 정보를 남기는 방식과 학습 target을 바꾼다.

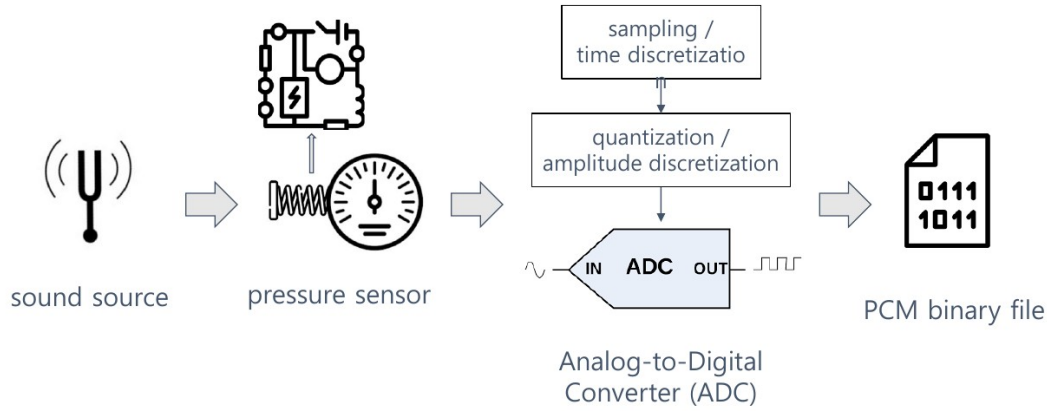
Stage	Input	Core question	Output
Digitize	pressure	언제·얼마나 측정?	PCM
Analyze	samples	어느 주파수가 언제?	STFT/Mel
Align	frames	token과 어떻게 대응?	CTC/LAS/RNN-T
Pre-train	unlabeled audio	target을 어디서 생성?	speech features

신호 처리에서는 보존 정보와 손실 조건을, ASR에서는 정렬과 출력 이력을 구분한다.
SSL에서는 transcript 없이 target을 만드는 방식을 비교한다.

Sampling and PCM

Sampling은 시간, quantization은 진폭을 이산화한다.

Analog and Digital Signals



The sinusoidal waveform follows from physics equations, see this slide
https://theory.labster.com/sound-waves-dbs/

5

Lecture 2, PDF p. 05 — analog pressure, sampling, quantization, PCM

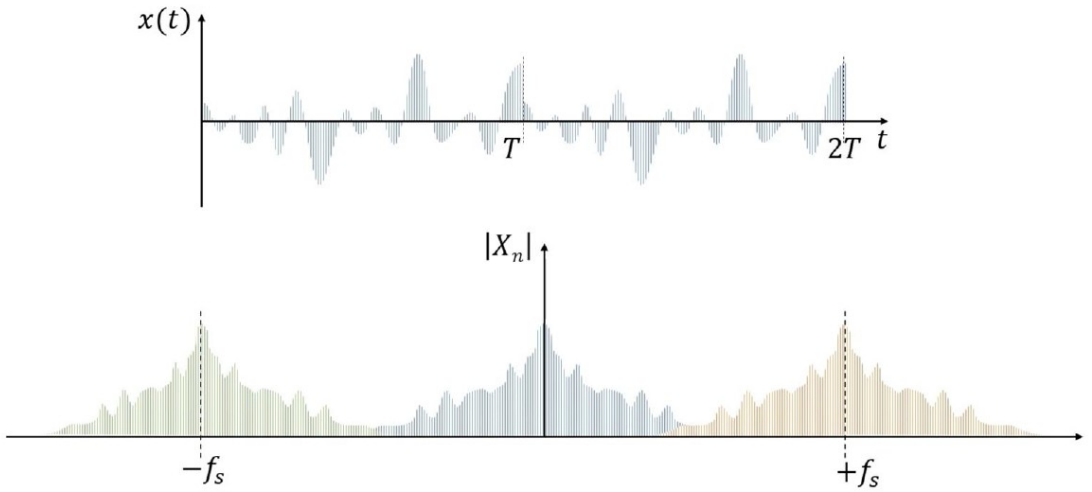
16-bit·44.1 kHz·stereo PCM의 payload는
$16 \times 44{,}100 \times 2 = 1{,}411{,}200$ bit/s다.

낮은 sampling rate와 낮은 bit depth는 서로 다른 정보를 잃는다.

Nyquist and Aliasing

Fourier basis는 표본의 주파수 성분을 분석한다. Aliasing 방지는 표본화 전에 결정된다.

Discrete-Time Fourier Series (DTFS)



What if the Nyquist sampling condition is not satisfied?

https://www.youtube.com/watch?v=nreiTseFZQ0

41

Lecture 2, PDF p. 37 — spectral replicas overlap when Nyquist is not satisfied

fs=8 kHz에서 zero-phase 5 kHz와 3 kHz cosine은 같은 표본을 만든다:
$\cos(2\pi \cdot 5n/8) = \cos(2\pi \cdot 3n/8)$.

표본화 전 anti-alias low-pass filter와 충분한 fs가 겹침을 막는다. FFT 길이나 window는 이미 잃은 원 주파수를 복원하지 못한다.

Orthogonal Fourier Bases

서로 다른 harmonic basis의 한 주기 inner product는 0이다.

Fourier Series: Exponential form

- Fourier Series & Fourier Coefficient
  - **Fourier Series:** exponential form

$$x(t) = \sum_{n=-\infty}^{\infty} c_n e^{2\pi i \frac{n}{P} t}$$

Frequency component (Frequency basis)

pure frequency component at $n\omega_0$
n=0 → 0 Hz (DC)
n=1 → f_0
n=2 → $2f_0$
…

- $\omega_0 = \frac{2\pi}{P}$
- Frequency basis:
  $\cdots, e^{-j2\omega_0 t}, e^{-j\omega_0 t}, 1, e^{j\omega_0 t}, e^{j2\omega_0 t}, \cdots$
- What is the basis?

$$e_1 = \begin{bmatrix}1\\0\\0\end{bmatrix} \quad e_2 = \begin{bmatrix}0\\1\\0\end{bmatrix} \quad e_3 = \begin{bmatrix}0\\0\\1\end{bmatrix}$$

$e_i^T e_i = 0 \ (i \neq j)$ — Orthogonal basis

- Frequency basis:

Inner product

$$\phi_n(t) = e^{jn\omega_0 t}$$

$$< \phi_n, \phi_m > = \int_0^P \phi_n(t)\phi_m^*(t)dt$$

$$= \int_0^P e^{jn\omega_0 t} e^{jm\omega_0 t} dt = \int_0^P e^{j(n-m)\omega_0 t} dt$$

$$< \phi_n, \phi_m > = \begin{cases} P, n \neq m \\ 0, n = m \end{cases}$$

28

Lecture 2, PDF p. 26 — complex exponential bases and one-period inner product

직교성은 inner product로 각 주파수 성분을 분리할 수 있다는 뜻이다.

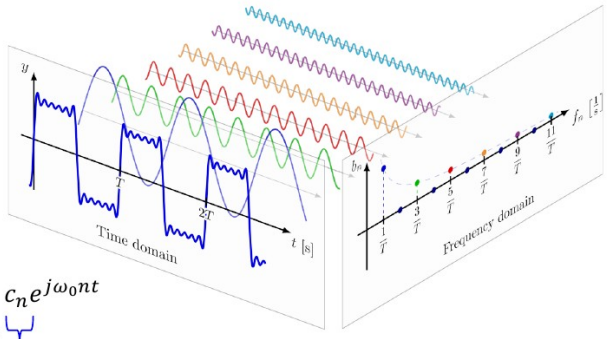
원문 빨간 식의 조건이 뒤바뀌었다: n=m이면 P, n≠m이면 0이다.

Fourier coefficient는 각 basis 방향의 성분을 측정한다.

Series to Transform

주기가 커지면 harmonic 간격이 좁아져 합이 적분으로 이어진다.

Fourier Transform



- Fourier Series & Fourier Coefficient

P: the period

$$c_n = \frac{1}{P}\int_{-P/2}^{P/2} x(t)e^{-2\pi i\frac{n}{P}t}dt \qquad x(t) = \sum_{n=-\infty}^{\infty} c_n e^{j\omega_0 nt}$$

Fourier coefficient

$$\omega_0 = \frac{2\pi}{P} \qquad \frac{1}{P} = \frac{\Delta\omega}{2\pi}$$

- Let's assume $P \to \infty \rightarrow \omega_0 \to 0 \rightarrow \Delta\omega = \omega_0 \to 0$

Frequency — Original signal

$$X(\omega) = \int_{-\infty}^{\infty} x(t)e^{-j\omega t}dt$$

Fourier transform

- Fourier Transform!
  - A mathematical formular that allows us to decompose any signal into its individual **frequencies** and the frequency's **amplitude**

https://en.wikipedia.org/wiki/Fourier_series

32

Lecture 2, PDF p. 29 — increasing period turns frequency sums into an integral

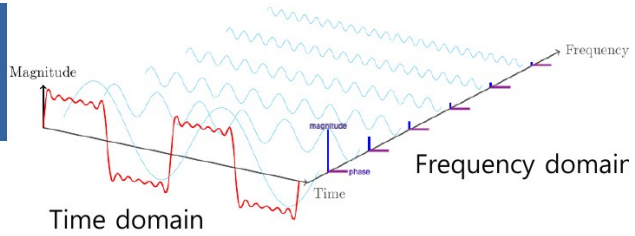
Fourier series는 주기 신호의 이산 harmonic 계수를, Fourier transform은 비주기 신호의 연속 spectrum을 표현한다.

정변환은 성분을 분석하고
역변환은 그 성분을 합성한다.

CTFS to DTFS

한 주기를 N번 표본화하면 harmonic index가 N을 주기로 같아진다.

Fourier Series → DTFS



- Fourier Series & Fourier Coefficient

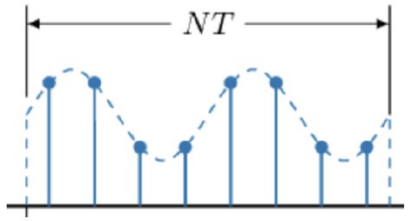
Fourier Series

$$x(t) = \sum_{n=-\infty}^{\infty} c_n e^{2\pi i \frac{n}{P} t}$$

Fourier coefficient

$$c_n = \frac{1}{P} \int_{-P/2}^{P/2} x(t) e^{-2\pi i \frac{n}{P} t} dt, \qquad \phi_k(t) = e^{jk\omega_0 t}$$

- Discrete-Time Fourier Series
  - In practice our time signal is time-limited and contains N non-zero samples taken with a sampling period T seconds.
  - Let's explore what happens when we try to find its Fourier coefficients.



https://ru.dsplib.org/content/dft/dft.html

36

Lecture 2, PDF p. 32 — periodic continuous signal and N uniform samples

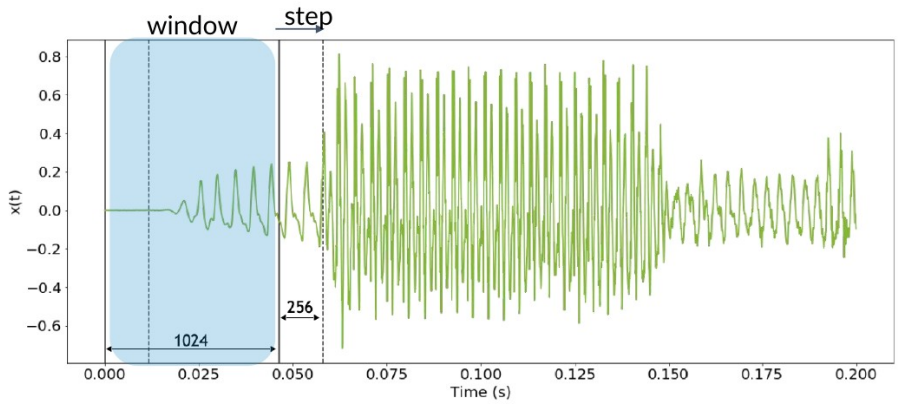
k와 k+qN은 표본 위에서 같은 basis이므로 CTFS 계수가 같은 나머지 class로 합쳐진다.

N〉2K와 제한된 harmonic bandwidth가 있어야 원래 계수를 겹침 없이 회수할 수 있다.

STFT Framing

STFT는 짧은 창을 이동하며 각 구간의 spectrum을 계산한다.

Shor-Time Fourier Transform



16 kHz, L=400, H=160은 25 ms 창과 10 ms hop이다.

1초 신호를 center padding 없이 처리하면 98 frame, NFFT=512의 one-sided spectrum은 257 bin이다.

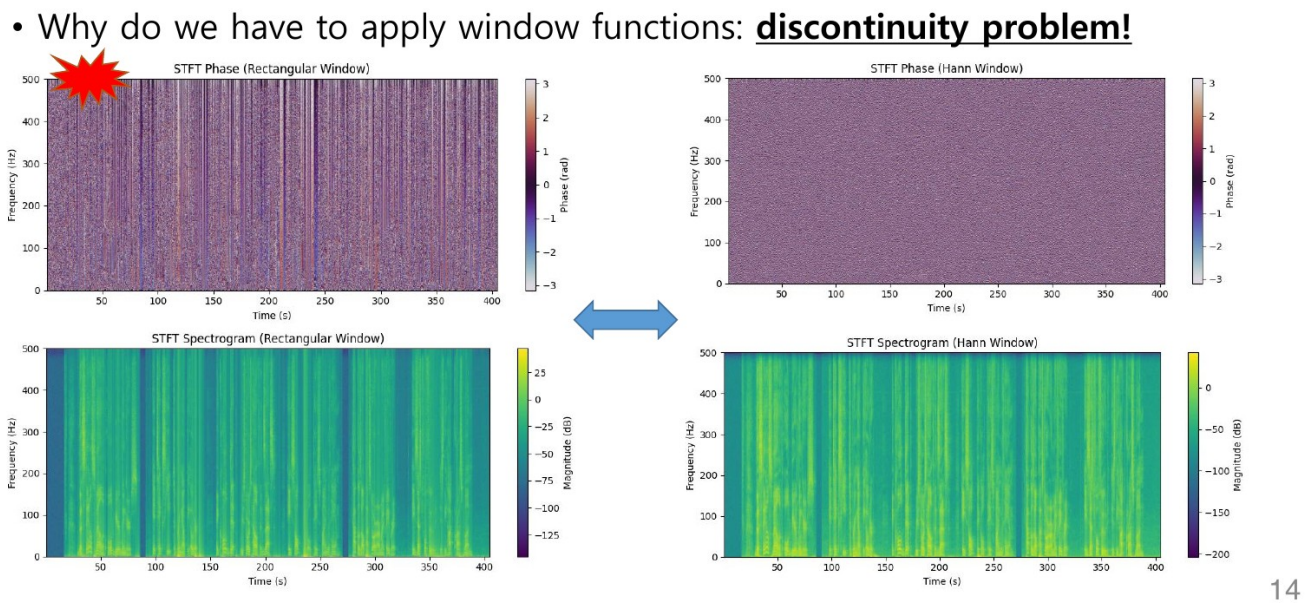
9

Lecture 2 Lab, PDF p. 09 — moving analysis window and overlap

Window Trade-offs

Window는 leakage를 없애지 않고 main-lobe와 side-lobe의 절충을 바꾼다.

Shor-Time Fourier Transform



Rectangular window는 경계 불연속 때문에 먼 sidelobe가 크다.

Hann window는 먼 leakage를 낮추지만 peak를 더 넓게 보이게 할 수 있다. 색 범위가 다르면 그림 밝기를 직접 비교하지 않는다.

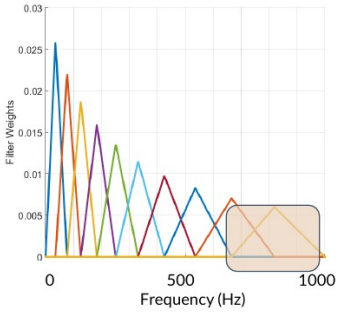
Lecture 2 Lab, PDF p. 14 — rectangular and Hann STFT views

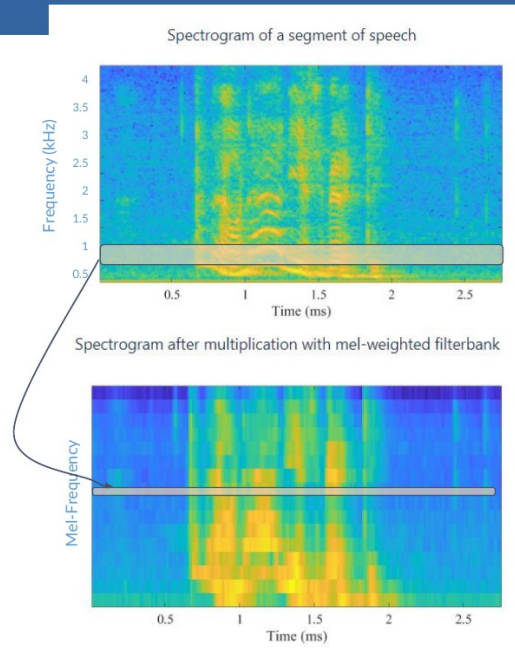
Mel Compression

Mel filterbank와 log는 서로 다른 두 압축이다.

Mel Spectrogram

- Mel Spectrogram





https://wiki.aalto.fi/display/ITSP/Cepstrum+and+MFCC

17

Lecture 2 Lab, PDF p. 17 — triangular Mel filters and Mel spectrogram

Filterbank는 여러 Hz bin을 band 별로 합쳐 주파수축을 줄인다. Log는 값의 dynamic range를 줄인다.

인식에는 유용하지만 phase와 세밀한 bin 정보를 잃으므로 원래 complex STFT를 일반적으로 복원할 수 없다.

Word Error Rate

WER은 편집 오류 수를 정답 단어 수로 정규화한다.

Symbol	Meaning	Example count
S	substitution	1
D	deletion	1
I	insertion	1
N	reference words	4

Reference 4단어에서 substitution 1, deletion 1, insertion 1이면 WER=(1+1+1)/4=0.75다.
Insertion이 많으면 WER은 1을 넘을 수 있다. 빈 reference에서는 별도 평가 규칙이 필요하다.

Byte Pair Encoding

BPE는 corpus에서 자주 붙는 token pair를 반복 병합한다.

DL-based ASR

- Subword Tokenization: BPE (Byte Pair Encoding)

Algorithm 1 Byte-pair encoding [30]

Input: set of strings D, target vocab size K
procedure BPE(D, K)
  $V \leftarrow$ all unique characters in D
  while $|V| < K$ **do** ▷ *Merge tokens*
    $t_L, t_r \leftarrow$ Most frequent bigram in D
    $t_{NEW} \leftarrow t_L + t_R$ ▷ *Make new token*
    $V \leftarrow V + [t_{NEW}]$
    Replace each occurrence of t_L, t_r in D with t_{NEW}
  end while
  return V
end procedure

aaabdaaabac

1. add Z <- aa
   ZabdZabac
2. add Y <- ab
   ZYdZYac
3. add X <- ZY
   XdXac

Example: low, lowest, newer → add "lo"=**A**, **A**w, **A**west, newer → ...
V: l, o, w, n, e, s, t, r — **V**: **A**, w, n, e, s, t, r

47

Training은 merge 순서를 만들고 저장한다. Inference는 새 문장에 같은 순서를 적용한다.

Subword는 출력 길이 U를 줄일 수 있지만 vocabulary V와 softmax 비용은 커질 수 있다.

Lecture 3, PDF p. 47 — BPE merge algorithm and toy sequence

CTC Blank and Paths

CTC blank는 frame 정렬을 허용하고 최종 전사에서는 제거된다.

Path	Merge repeats	Drop blank	Output
L ε L	L ε L	LL	LL
L L	L	L	L
ε a b	ε a b	ab	ab
a a b	a b	ab	ab

Collapse는 연속 중복을 먼저 합친 뒤 blank를 제거한다.
따라서 L ε L은 LL이지만 L L은 L이다. 같은 label을 두 번 출력하려면 사이 blank가 필요하다.

CTC Probability

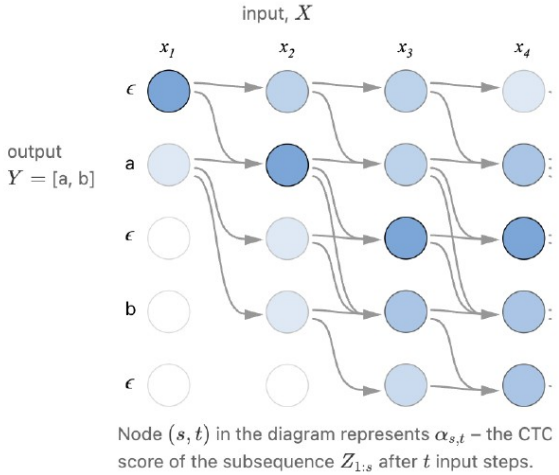
한 문자열의 확률은 그 문자열로 collapse되는 모든 경로 확률의 합이다.

DL-based ASR

- Efficient CTC Loss Compute: Example
  - Output $Y = [a, b]$

t	t=1	t=2	t=3	t=4
ε	0.6	0.2	0.3	0.4
a	0.3	0.5	0.2	0.1
b	0.1	0.3	0.5	0.5

O: probability distribution



33

각 frame의 분포는 encoder logit의 softmax에서 나온다. 한 경로 확률은 선택한 frame 확률의 곱이다.

서로 배타적인 유효 경로를 더한 뒤 −log를 취해 CTC loss를 얻는다.

Lecture 3, PDF p. 33 — frame distributions and valid CTC paths

CTC Forward DP

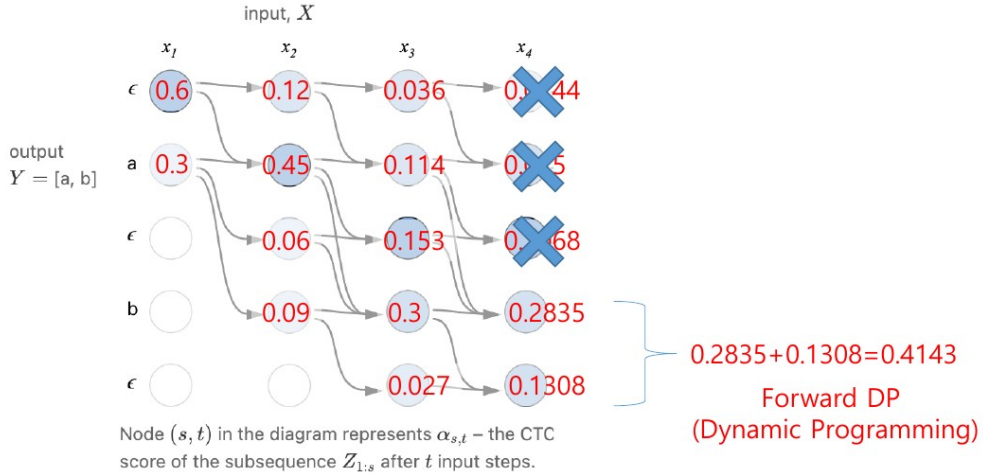
각 칸은 해당 상태에서 끝나는 모든 유효 부분 경로의 확률 합이다.

DL-based ASR

t	t=1	t=2	t=3	t=4
ε	0.6	0.2	0.3	0.4
a	0.3	0.5	0.2	0.1
b	0.1	0.3	0.5	0.5

O: probability distribution

- Efficient CTC Loss Compute: Example
  - Output $Y = [a, b]$



Node (s, t) in the diagram represents $\alpha_{s,t}$ – the CTC score of the subsequence $Z_{1:s}$ after t input steps.

35

Lecture 3, PDF p. 35 — forward probabilities and two terminal states

Stay·한 칸 전진·허용된 두 칸 skip 의 이전 합에 현재 frame 확률을 곱한다.

P(ab|X)=0.2835+0.1308=0.4143. 유한 경로의 합이며 극한 수렴이 아니다.

원문 합산식의 0.1038은 0.1308 의 오기다.

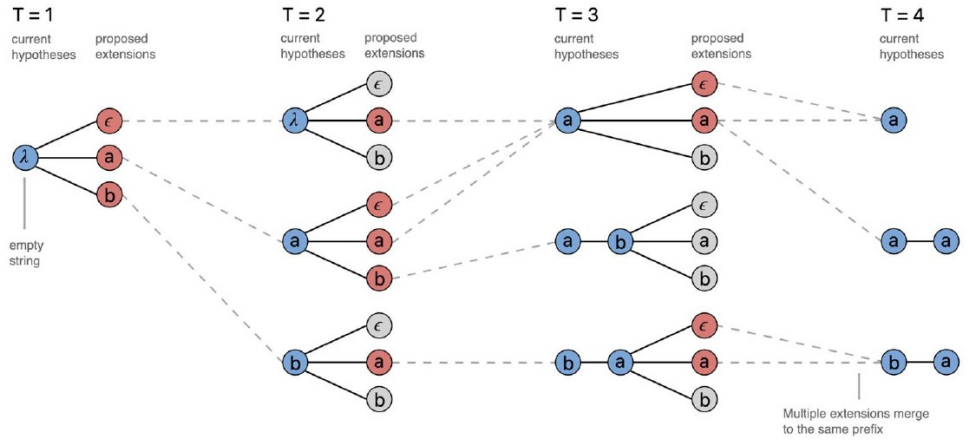
Greedy vs. Beam Search

최대 확률 경로가 최대 확률 문자열과 같다는 보장은 없다.

DL-based ASR

• CTC Beam Search (Beam size=3)

Beam search step:
- expand beam
- merge paths
- truncate beam



The CTC beam search algorithm with an output alphabet $\{\epsilon, a, b\}$ and a beam size of three.

40

Lecture 3, PDF p. 40 — prefix beam expansion and pruning

Greedy는 frame마다 한 token만 남긴다. Prefix beam search는 여러 prefix의 blank·non-blank 확률을 따로 누적한다.

유한 beam은 낮은 후보를 버리는 근사이며, 외부 LM을 쓰면 점수 scale과 length bonus도 확인한다.

Deep Speech 2 vs. Conformer

Encoder 구조와 정렬 loss는 같은 선택 축이 아니다.

Model	Local pattern	Long context	Objective / role
Deep Speech 2	convolution	recurrent layers	CTC objective
Conformer	conv module	self-attention	encoder architecture
CTC	—	—	path marginalization

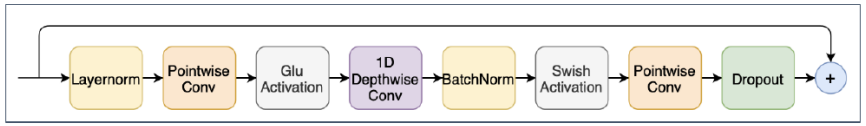
Deep Speech 2는 convolution과 recurrent layers로 음향 표현을 만들고 CTC로 정렬한다.
Conformer는 self-attention의 넓은 문맥과 convolution의 국소 패턴을 한 encoder block에 결합한다. CTC와 함께 쓸 수 있다.

Conformer Convolution

Depthwise convolution은 채널별 시간 이웃을 처리하고 pointwise convolution은 채널을 섞는다.

DL-based ASR

- Encoders: Conformer
  - **Conformer: Convolution-augmented Transformer for Speech Recognition, Anmol Gulati, Google, 2020**



Convolution module

GLU가 전달할 채널 정보를 조절하고 residual이 입력과 변환 결과를 합친다.

Subsampling stride 2를 두 번 쓰면 시간 길이는 대략 T/4지만 kernel·padding에 따라 정확한 길이는 달라진다.

Lecture 3, PDF p. 44 — Conformer convolution module

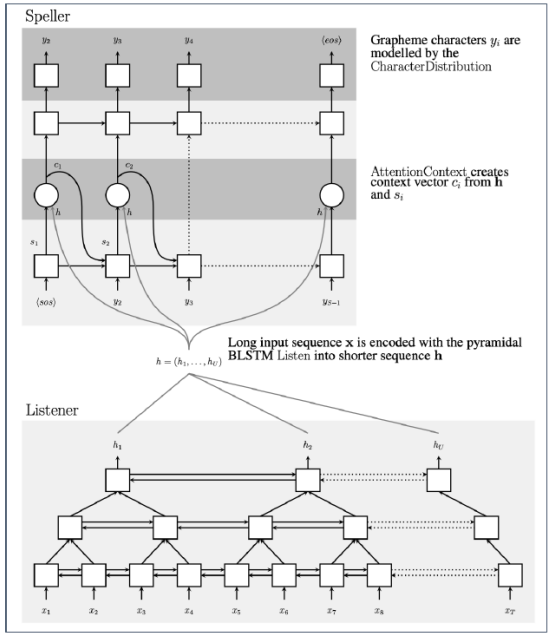
Listen, Attend and Spell

LAS는 출력 단계마다 필요한 입력 위치를 attention으로 다시 고른다.

Listen, Attend and Spell (LAS)

- **Listen, Attend and Spell**, William Chan et al., Carnegie Mellon University, Google Brain, 2015
  - The network produces character sequences without making any independence assumptions between the characters.
  - Modules:
    - Listener - encodes information about the input audio
    - Attend - tells Speller what parts of input are relevant during the current decoding step
    - Speller- decodes the latent representation into transcription

https://arxiv.org/abs/1508.01211



15

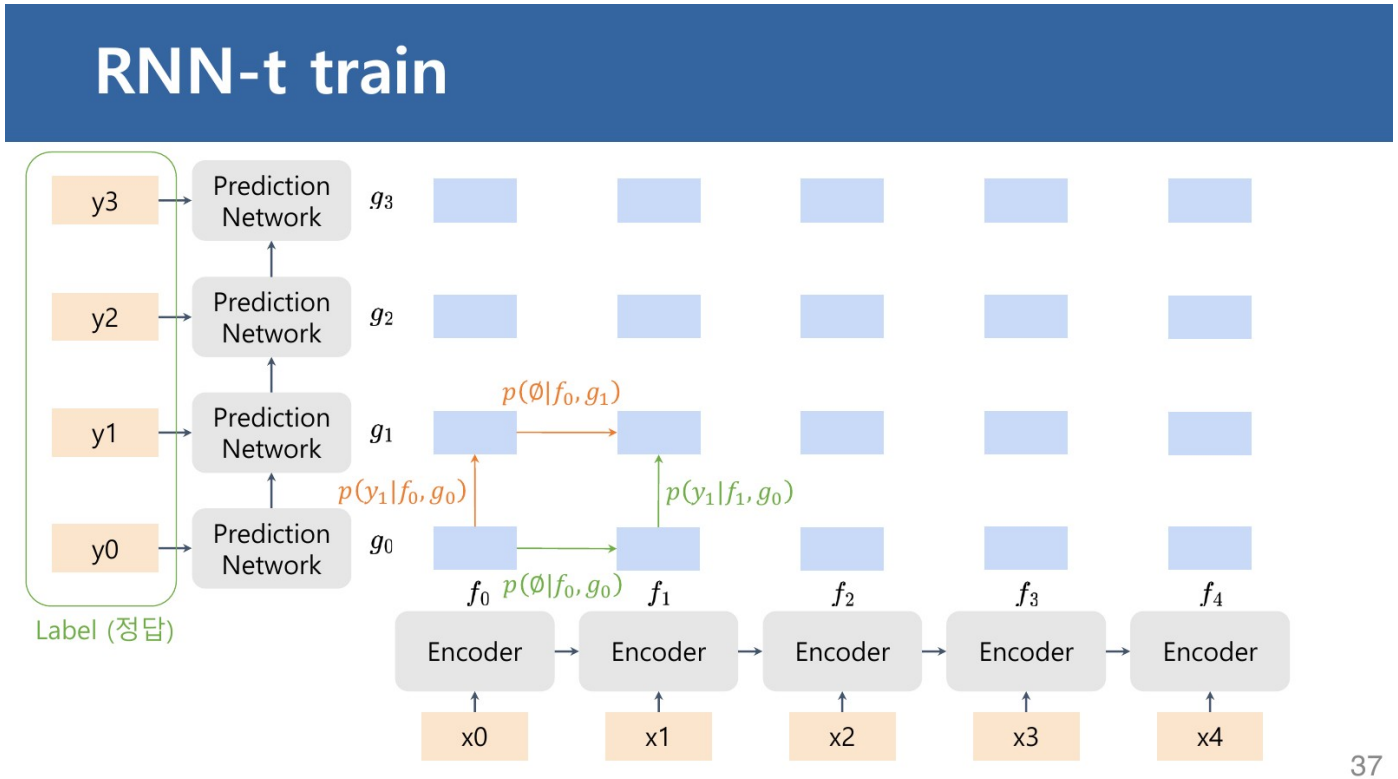
Lecture 4, PDF p. 15 — pyramidal listener, attention context, autoregressive speller

Listener는 음향 frame을 인코딩하고 Attend는 가중합 context를 만든다. Speller는 context와 이전 token으로 다음 token을 생성한다.

학습은 정답 prefix를, 추론은 생성 prefix를 쓰므로 오류가 누적될 수 있다.

RNN-T Lattice

RNN-T는 acoustic time과 output prefix를 두 축으로 둔다.



Blank는 (t,u)에서 (t+1,u)로 시간을 진행한다. Token은 (t,u+1)로 출력 길이를 늘린다.

Prediction network가 이전 출력 이력을 표현하므로 같은 frame에서도 여러 token을 낼 수 있다.

Lecture 4, PDF p. 37 — encoder and prediction-network axes in RNN-T training

CTC, LAS, and RNN-T

세 모델은 같은 정렬 문제를 서로 다른 확률 구조로 푼다.

CTC vs. LAS vs. RNN-t

Method	CTC	LAS	RNN-t
Streaming	yes	no	yes
Context	no	yes	yes
Argmax Complexity	O(enc + dec)	O(enc + T * dec)	O(enc + T * dec)
Beam Search Complexity	O(enc + dec + T * bs)	O(enc + T * bs * dec)	O(enc + T * bs * dec)

CTC는 출력 이력 없이 frame 경로를 합한다. LAS는 attention과 autoregressive prefix를 쓴다. RNN-T는 prefix와 시간축 경로를 함께 합한다.

Streaming 여부는 encoder 문맥과 실제 추론 설정까지 확인해야 한다.

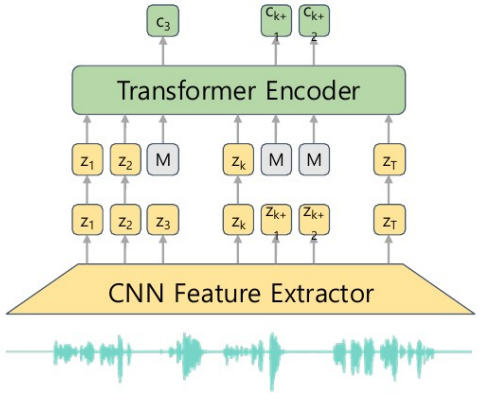
44

Lecture 4, PDF p. 44 — lecture comparison of CTC, LAS, and RNN-T

wav2vec 2.0 Context

가린 latent 위치를 주변 음성 문맥으로 표현한다.

wav2vec2.0



24

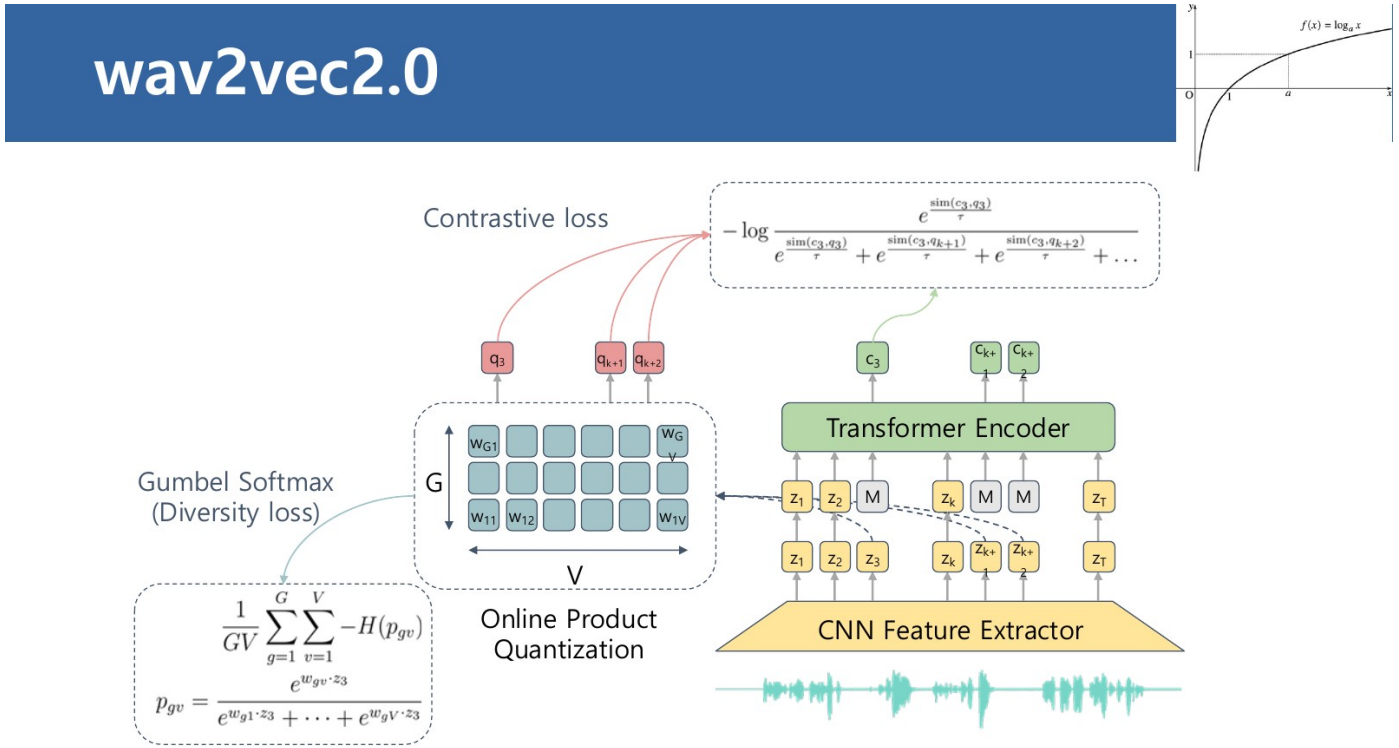
CNN이 waveform을 z_t로 바꾸고, 일부 위치를 mask embedding으로 치환한 뒤 Transformer가 c_t를 만든다.

Masking은 spectral mask로 음원을 복원하는 연산이 아니라 사전학습 문제를 만드는 절차다.

Lecture 5, PDF p. 24 — CNN latent sequence, masked positions, Transformer context

wav2vec 2.0 Targets

Context c_t는 같은 위치의 quantized target q_t를 distractor보다 잘 골라야 한다.



31

가리지 않은 z_t는 product quantizer를 거쳐 q_t가 된다. Contrastive loss는 positive와 후보를 구별한다.

Diversity objective는 일부 code entry만 쓰는 collapse를 줄인다. q_t는 문자 label이 아니다.

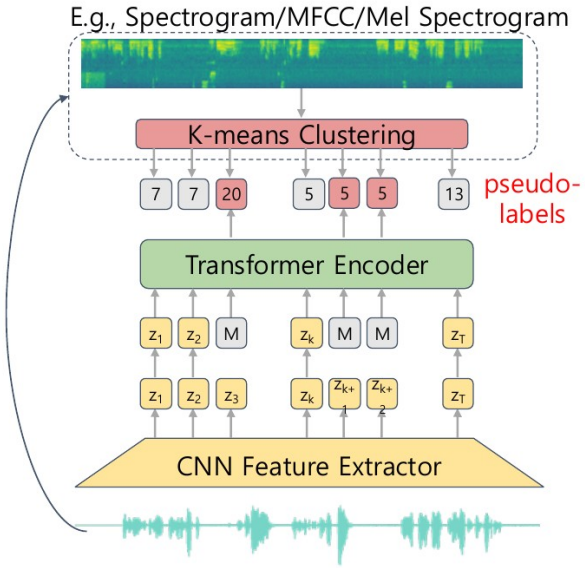
Lecture 5, PDF p. 31 — online quantization, contrastive loss, diversity loss

HuBERT Hidden Units

HuBERT는 offline clustering으로 만든 ID를 가린 위치에서 예측한다.

HuBERT

- HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units (Meta, 2021)
- **H**idden **U**nit **BERT**



46

첫 target은 MFCC 등의 feature를 k-means로 묶어 만든다. 7·20·5는 cluster ID이며 음소 이름이나 크기 순서가 아니다.

이전 HuBERT layer를 다시 clustering해 target을 개선할 수 있다.

Lecture 5, PDF p. 46 — offline k-means pseudo-labels and masked prediction

Final Recap

표현, encoder, 정렬 목표는 서로 다른 설계 축이다.

Question	Method	Answer
무엇을 보존?	STFT/Mel	time-frequency / compressed bands
어떻게 정렬?	CTC/LAS/RNN-T	paths / attention / 2-D lattice
어떻게 인코딩?	DS2/Conformer	RNN / attention+conv
target은 어디서?	wav2vec 2.0/HuBERT	quantizer / clustering

FFT는 DFT 계산법이고, Mel은 지각적 축약이다. Conformer는 encoder이며 CTC는 정렬 목표다.
wav2vec 2.0은 online quantized target, HuBERT는 offline cluster target을 쓴 뒤 labeled ASR로 fine-tuning 한다.